Predicting Whether a Clinical Trial Meets Its Primary Endpoint

CS5785 / ORIE5750 / ECE5414 — Midterm Project, Part 2

Anirudha Hadagali
Cornell University

Michael Mazzola
Cornell University

Tarun Mandalapu
Cornell University

Abstract

We predict whether a clinical trial meets its primary endpoint, separately for Phase I, II, and III. The score is log loss, so a confident wrong probability is punished more than a near miss. The test trials start later than the training trials, and later trials succeed more often.

We started from eligibility text and logistic regression, then compared trees, drug features, and phase-specific fits. The model we kept is a stack of text and drug classifiers, with more weight on recent trials, blended with a neighbor model that also prefers recent similar trials. Our earlier text-and-tree ensemble still made different mistakes, so the submission is 70% of that recent-neighbor stack and 30% of the earlier ensemble. The reported Kaggle log loss is 0.58465. A later blend looked better on our historical checks and scored 0.58492, so we kept the first of those two.

Team responsibilities. All three authors worked on Part 1, Part 2, and the report. **Tarun Mandalapu** built the first Part 2 classifier. **Anirudha Hadagali** led the later tuning that produced the submitted blend. **Michael Mazzola** led the Part 1 model. Tarun Mandalapu wrote the combined project report. This Part 2 write-up follows the executed notebooks and the later submission checks.

1. Methods

1.1. What we are predicting

Each row is one trial in one phase. A trial that spans two phases appears twice, with the same success label, and those two rows always stay on the same side of a split. Success means the trial met its primary endpoint. The training file has 8,037 rows from trials that started through 2013. The validation file has 1,013 rows from the same years, split by trial so a trial is never in both. The test file has 1,500 rows, 500 in each phase, from 2013 to 2022. None of those trials appear in the other files.

Kaggle scores the average of the three phase log losses. Because each phase has the same number of test rows, that is the same as the overall log loss. We therefore select models by the equal average of the Phase I, II, and III log losses, not by a single F1 number. F1 at a cutoff of 0.5, ROC-AUC, and precision-recall AUC are reported as diagnostics. Trial IDs are used only to keep rows together and to match Part 1. They are not inputs. Part 1 reason labels and reason embeddings are never given to the outcome model.

1.2. The first models

The baseline is a logistic regression on eligibility-criteria text. We turn the criteria into word and bigram counts, keep at most 35,000 terms that appear in at least three trials, and use sublinear term frequency. Three phase indicators let one model share text evidence across phases. The regularization is $C = 2$. A separate classifier for each phase was also tried. Phase I has many fewer rows, so a phase-only fit overfits it.

Around that baseline we compared disease and drug text, ICD codes, drug targets, actions, and pathways [2], Morgan fingerprints [1, 4], naive Bayes, nearest neighbors, and tree ensembles. Trees can represent interactions that a linear text model misses. Fingerprints did not reliably help, and a matched drug is less common in the test file (39%) than in training (46%), so a model that leans on drug structure is exposed to a coverage shift. Naive Bayes could rank trials, but its probabilities were far too sharp for a log-loss contest.

Our first submitted ensembles mixed logistic text models with CatBoost trees. The version with the best local score put more weight on phase-specific pieces and scored 0.59562 on Kaggle. Reducing the tree share in Phases II and III improved that to 0.58993. The local winner was not the leaderboard winner. That was the first sign that a score on the random validation split was a weak guide to a later test cohort.

1.3. The stack, and why recent trials count more

The stronger reference is a stack of eight models [3, 5]. Separate logistic regressions read eligibility text, disease text, drug names, codes, drug annotations, and simple trial metadata. A naive Bayes model and a nearest-neighbor model

also read the eligibility text. A second logistic regression learns how much to trust each of the eight. So that this combiner does not see predictions that were fit on the answer, each base model is scored on held-out folds: five folds, with every row of a trial kept together. The neighbor model uses the 50 closest trials.

Trials get more successful over time, and every test trial starts later than the training trials. We therefore give recent trials more weight. The weight decays exponentially with a three-year time constant, so a trial loses about half its weight every two years. The same weights go on the base models and on the combiner. The decay was chosen on time-ordered holdouts, not on the leaderboard. This recency-weighted stack, refit on all 9,050 labeled rows, is the reported 0.58688 submission. An otherwise identical stack without the recency weights scored 0.58810. Our first submissions had been fit on the 8,037 training rows only, so part of the earlier gap was the extra labeled data, not only the architecture.

1.4. Neighbors that notice time

The neighbor piece inside that stack ignored recency. It used raw text similarity and pulled every estimate toward a 50% success rate. We tested sharper similarity, neighborhoods of several sizes, similarity that also uses disease, drug, and ICD text, and a prior estimated from the training rows of the same phase.

Two versions mattered. One replaces only the neighbor input and then refits the stack. The submitted neighbor version does not replace the original stack. It averages that stack, at 50%, with a second stack whose recency decay is faster (a two-year time constant) and which also receives eight neighborhood estimates. The average is fixed. It is not fit on the validation labels. This is file 10. It improved every time-ordered window we had declared in advance. Its random validation log loss got slightly worse. Kaggle moved from 0.58688 to 0.58609.

1.5. Why the submission mixes in the older ensemble

File 10 was better on later years and worse on the random grouped checks. Our older text-and-tree ensemble had the opposite pattern, so the two errors were not the same. The older ensemble is half a mixture of three logistic models, a quarter a depth-6 CatBoost model, and a quarter a depth-8 CatBoost model. We blended probabilities rather than merging the features.

We tried 20% of the older ensemble first, because a 30% blend had a worse worst-period score in the local checks. That 20% blend, file 11, scored 0.58474. Raising the older share to 30%, file 13, scored 0.58465. The second gain was only 0.00009, so we did not keep adding more of the older model. A later blend that also mixed in a recent tree and a phase trend looked better on the historical windows and scored 0.58492, which is worse. With one upload left and first place already reported, we did not submit the last prepared file. Its local gains were small and its grouped score was slightly worse.

The submitted probability is

$$0.70\,P_{10}+0.30\,P_{\text{early}}, \quad P_{10} = 0.50\,P_{\text{recency}}+0.50\,P_{\text{fast neighbors}}.$$

After the recipe was fixed, every supervised piece was refit on all 9,050 labeled rows. Test outcome labels are never used. In the stack inherited from the 0.58688 model, the word lists and IDF weights are fit on training, validation, and unlabeled test text. That changes the features, not the success labels. The older ensemble fits its own word lists only on the rows it is allowed to learn from.

1.6. How a candidate was allowed to advance

Every later comparison uses the same saved trials. The supplied validation split fits on 8,037 rows and scores 1,013. Three time-ordered windows train through 2007, 2009, or 2011 and score the following years. A grouped cross-validation pools the 9,050 labeled rows and holds out trials, twice, with three folds. Three longer gaps were scored only after a recipe was frozen. Word lists are rebuilt inside each fit, including the inner stacking folds. We required a gain on the time-ordered average, and we recorded the worst window and the grouped score even when they went the wrong way. A higher F1 at 0.5 was not enough.

1.7. The reason-error check

After the outcome model is fit, we look up each failed validation trial in Part 1. If Part 1 has a real reason label, we use it. Otherwise we use a frozen Part 1 prediction. Trials that ran to completion are absent from Part 1 and form their own "no recorded stopping reason" group. A trial with several reasons is counted under each. Phases are separate, so a two-phase trial is not counted twice inside one phase. The error rate is the share of these failed trials given a success probability of at least 0.5. This table is a diagnosis. It is not a feature and it is not the selection score.

2. Results

Table 1 is the decision path. Lower log loss is better. The local numbers are the equal average of the three phase losses on the shared checks. File 13 is the submitted model. It is the best reported Kaggle score. File 15 won the local time-ordered average and lost on Kaggle.

Table 2 is the supplied validation split, under preprocessing fit only on the training rows of that split. The baseline is the criteria logistic model. The recency stack is the 0.58688 reference. File 10 is the neighbor blend before the older ensemble is added. File 10 does not win this table. Its gain showed up on the later-year windows and on Kaggle, which is why we did not select on validation F1.

Model	Time	Grouped	Kaggle
Phase-specific ensemble	—	—	0.59562
Text-heavy ensemble	—	—	0.58993
Stack, no recency weights	0.5421	0.5797	0.58810
Recency stack	0.5374	0.5874	0.58688
File 10, recent neighbors	0.5359	0.5869	0.58609
File 11, 20% early ensemble	0.5339	0.5812	0.58474
File 13, 30% early ensemble	0.5334	0.5790	**0.58465**
File 15, tree and trend added	**0.5327**	**0.5783**	0.58492

Table 1. Later models on the shared checks. Time is the average of the original forward windows. Grouped is repeated trial-grouped cross-validation. The first two Kaggle scores are early submissions that used a different local protocol, so those two local cells are left blank. Kaggle scores are the reported public scores.

Phase	Metric	Baseline	Recency	File 10
I	F1	0.765	**0.793**	0.762
	ROC-AUC	0.783	**0.794**	0.787
	PR-AUC	**0.835**	0.830	0.824
	log loss	0.556	0.556	0.560
II	F1	**0.657**	0.647	0.638
	ROC-AUC	0.740	0.753	**0.754**
	PR-AUC	0.717	**0.732**	0.730
	log loss	0.600	0.593	0.593
III	F1	0.833	0.825	**0.829**
	ROC-AUC	0.774	**0.790**	0.786
	PR-AUC	0.862	**0.882**	0.877
	log loss	0.529	**0.516**	0.521

Table 2. Validation metrics by phase. F1 uses a cutoff of 0.5. ROC-AUC, PR-AUC, and log loss use the predicted probability that the trial succeeds. Phase sizes are 149, 501, and 363 rows. Best value per row in bold. Phase I log loss ties at three decimals; the recency stack is slightly lower.

Table 3 is the reason check on the failed validation trials. The baseline is the criteria logistic model. The ensemble is the early text-and-tree model that later became the 30% share of file 13. Most reason tags are Part 1 predictions, not confirmed labels. The groups large enough to read are Phase II Recruitment and the three no-reason groups. Phase III failures are called successful far more often than Phase II failures. Adding clinical context lowers some of those rates and raises others. The later neighbor model did not fix the hardest cell: all 9 Phase III efficacy-tagged failures were still called successful.

3. Discussion

Log loss and F1 answer different questions. File 10 lowers log loss on later years while its validation F1 goes down in Phases I and II. That is not a failed model. F1 cuts every probability at 0.5. A model that pulls uncertain trials toward the recent success rate can cross that cutoff the wrong way and still assign the better probability. The contest scores the probability. We treated F1 as a check, not as the thing to maximize. Had F1 been the target, the right next step would have been a separate cutoff for each phase.

Phase	Reason	n	Base	Ens.
I	Safety	9	.56	.44
I	Funding	4	.50	.50
I	Efficacy	2	.50	.50
I	Recruitment	22	.36	.27
I	Other	5	.80	.80
I	No reason	28	.43	.39
II	Safety	15	.40	.40
II	Funding	4	.00	.25
II	Efficacy	10	.30	.30
II	Recruitment	87	.20	.16
II	Other	18	.33	.28
II	No reason	144	.35	.34
III	Safety	6	.50	.67
III	Funding	0	—	—
III	Efficacy	9	.89	.67
III	Recruitment	21	.52	.57
III	Other	8	.63	.75
III	No reason	82	.56	.55

Table 3. Share of failed validation trials predicted successful, by phase and stopping reason. "No reason" means the trial never appears in Part 1. A trial with several reasons is listed in each. Funding has no Phase III failures.

The test set is a later cohort. Validation trials are drawn from the same years as the training trials. Test trials start later, and success becomes more common over time. A model fit to the average training year is too pessimistic about the test file. Recency weights are the direct answer: a recent trial counts more, with a decay chosen on time-ordered holdouts. The neighbor change does the same job from the other side. A similar past trial is more relevant if it is also recent, and a Phase I trial should be compared with Phase I. File 10's validation log loss is worse than the plain recency stack, and its Kaggle loss is better. We trusted the check that looks like the test set.

Why 30% of the old ensemble, and then stop. The older ensemble is weaker on later years and stronger when the years are mixed. Blending it with file 10 is useful only while the two models miss different trials. The first 20% was worth 0.00135 on Kaggle. The next 10% was worth 0.00009. File 15 added a recent tree and a phase trend, improved the local averages, and lost 0.00027 on Kaggle. After that, a better historical average was not a reason to upload. The last

prepared file had small local gains, intervals that included zero, and a slightly worse grouped score. We left the last submission slot unused.

Where the errors stay. Failed Phase III trials are hard to recognize. They often ran to the end and simply missed the endpoint, so the eligibility text looks like the text of a success, and the high Phase III success rate pulls the probability up. Recruitment and funding failures are more administrative, and the early ensemble catches a few more of them. The Phase III efficacy row moves from 8 of 9 to 6 of 9 in Table 3, then back to 9 of 9 under the neighbor model. All nine of those tags are Part 1 predictions. Safety, Funding, and Other are small in every phase. Only Phase II Recruitment and the three no-reason groups are large, and membership in the no-reason group is a lookup rather than a prediction. The reason table told us to look past eligibility text. It was not a safe place to tune a cutoff.

What we would not do again. We would not ship the model with the best random validation loss. File 03 did that and scored 0.59562. We would not add fingerprints or a raw year slope just because the feature is available. Year fitted on 2000–2013 has to be believed out to 2022, and drug structure is missing more often in the test file than in training. We would not treat a 0.0001 local gain as a new submission once the leaderboard had already contradicted that kind of gain.

References

[1] Greg Landrum et al. RDKit: Open-source cheminformatics. https://www.rdkit.org. Accessed November 2026. 1

[2] Marija Milacic, Deidre Beavers, Patrick Conley, et al. The Reactome pathway knowledgebase 2024. *Nucleic Acids Research*, 52(D1):D672–D678, 2024. 1

[3] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. 1

[4] David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50 (5):742–754, 2010. 1

[5] David H. Wolpert. Stacked generalization. *Neural Networks*, 5 (2):241–259, 1992. 1